

Reviewer Pack — Minimal Rank of Toric Varieties over Boolean Functions vs Mo...

Entry 36ff10223572 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 10:04:54 UTC

Minimal Rank of Toric Varieties over Boolean Functions vs Monotone Circuit Width

Entry ID: 36ff10223572 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 10:04:54 UTC

1. Conjecture

Field A (mathematical branch): Toric Geometry **Field B** (complexity object): Complexity Theory

Statement:

[‘For a given 3-CNF formula F with n variables, let V be the associated real algebraic variety defined by the set of clauses in F over the Boolean ring. Then, the minimal rank of V is upper bounded by the width of any monotone circuit that decides F .’, ‘Equivalently: For all 3-CNF formulas F with n variables, there exists a constant c_F such that the minimal rank of the toric variety associated with F is at most c_F times the width of the monotone circuit deciding F .’]

Rationale (proposer’s reasoning):

[‘Toric geometry provides a combinatorial description of real algebraic varieties, which could potentially expose structural properties of boolean functions. The conjecture suggests that such properties are related to the complexity of computing them, specifically through monotone circuits, which are known to be related to SAT and other complexity classes.’, ‘If true, this would establish a connection between the geometric structure of boolean functions and their computational complexity, providing new insights into both fields.’]

Taxonomy category: toric_geometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dd782e290eb64c20

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each 3-CNF formula F , if the ratio of the minimal rank of the associated toric variety V to the monotone circuit width c_F times is less than or equal to a constant factor C for all seeds, and no seed produces a ratio greater than C .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - toric geometry AND complexity theory AND 'minimal rank of varieties' AND 'Boolean functions' AND 'monotone circuit width' - algebraic variety in toric geometry AND 3-CNF formula AND 'monotone circuit' AND 'circuit width' AND 'Boolean ring' - 'real algebraic variety' AND 'toric variety' AND 'complexity theory' AND 'minimal rank' AND 'monotone circuit decision'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n):
        clauses = []
        for _ in range(2 * n):
            clause = [random.choice([f'x{i+1}', f'-x{i+1}']) for i in range(random.randint(1, 3))]
            clauses.append(clause)
        return clauses

    def construct_toric_variety(clauses):
        toric_variety = []
        for clause in clauses:
            point = [0] * (len(clause) + 1)
            for literal in clause:
                if literal[0] == 'x':
                    var = int(literal[1:]) - 1
                else:
                    var = int(literal[2:]) - 1
                point[var] = 1
            toric_variety.append(point)
        return toric_variety

    def compute_minimal_rank(toric_variety):
        n = len(toric_variety[0])
```

```

rank = 0
for i in range(n):
    if any(row[i] == 1 for row in toric_variety):
        rank += 1
return rank

def compute_monotone_circuit_width(clauses):
    # Simplified DPLL solver to estimate width
    def dpll(clauses, assignment):
        if not clauses:
            return True
        literal = next(l for l in clauses[0] if l not in assignment and -l not in assignment)
        for value in [True, False]:
            new_assignment = assignment.copy()
            new_assignment[literal] = value
            if dpll(clauses, new_assignment):
                return True
        return False

    width = 0
    for literal in set(l for clause in clauses for l in clause):
        positive_clauses = [clause for clause in clauses if literal in clause]
        negative_clauses = [clause for clause in clauses if -literal in clause]
        width = max(width, len(positive_clauses), len(negative_clauses))
    return width

n = random.randint(5, 40)
clauses = generate_3cnf(n)
toric_variety = construct_toric_variety(clauses)
minimal_rank = compute_minimal_rank(toric_variety)
circuit_width = compute_monotone_circuit_width(clauses)

if circuit_width == 0:
    return {
        "metric_name": "minimal_rank_over_circuit_width",
        "metric_value": float('inf'),
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "circuit_width_zero"
    }

ratio = minimal_rank / circuit_width
return {
    "metric_name": "minimal_rank_over_circuit_width",
    "metric_value": ratio,
    "instances_tested": 1,
    "conjecture_holds": True if ratio <= 2 else False, # Placeholder constant factor C=2
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.randint(1000, 9999) for _ in range(30)]

    results = []
    for seed in seeds:

```

```

trial_result = run_trial(seed)
print(f"TRIAL: {trial_result}")
results.append(trial_result)

mean_value = sum(result["metric_value"] for result in results) / len(results)
std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='not_supported' first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_97b75864.py", line 99, in <module>
    trial_result = run_trial(seed)
                    ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_97b75864.py", line 72, in run_trial
    minimal_rank = compute_minimal_rank(toric_variety)
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_97b75864.py", line 45, in compute_minimal_rank
    if any(row[i] == 1 for row in toric_variety):
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_97b75864.py", line 45, in <genexpr>
    if any(row[i] == 1 for row in toric_variety):
        ~~~^^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with appropriate error handling and debugging to ensure it completes successfully.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12129
2	preregistration	ollama_remote	glm4:latest	0	0	5393
3	novelty	ollama_remote	glm4:latest	0	0	5278
4	novelty	ollama_remote	glm4:latest	0	0	5222
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14704
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6491
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13685
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11284
9	judge	ollama_remote	glm4:latest	0	0	8836

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 83023 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/36ff10223572.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/36ff10223572.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/36ff10223572.tar.gz` (if generated)